

Student Learning Management System

Architecture Document

Upgrade2Architect Phase 3 — Version 1.0

AWS | Java Spring Boot | React/Next.js | Docker | ECS Fargate | PostgreSQL | Redis | DynamoDB | S3

Attribute	Value
Document title	Student LMS — Architecture Document
Version	1.0
Status	Drafts
Technology stack	AWS, Java Spring Boot 3.x, React/Next.js, Docker, ECS Fargate
Author	Pratyush Chandra Jha -2154615
Date	22-May-2026
Classification	Confidential
Parent documents	HLD_Student-Learning-Management-System_2154615_v1.0.docx LLD_Student-Learning-Management-System_2154615_v1.0.docx API-Design_Student-Learning-Management-System_2154615_v1.0.docx

Table of Contents

Table of Contents 2

1. Architecture Overview 3

 1.1 System description 3

 1.2 Architecture style 3

 1.3 Architecture layers 3

2. AWS Component Reference 5

 2.1 Compute 5

 2.2 Networking and edge 5

 2.3 Messaging and eventing 5

 2.4 Data stores 6

 2.5 Security services 6

 2.6 DevOps and observability 7

3. Microservice Catalogue 8

4. Key Data Flows 9

 4.1 Student submits a quiz (happy path) 9

 4.2 Certificate generation flow 9

5. Architecture Design Decisions 11

6. Non-Functional Requirements Compliance 13

7. Scalability and High Availability 14

 7.1 Scalability approach 14

 7.2 High availability 14

8. Security Architecture 15

 8.1 Defence in depth layers 15

9. Operational Considerations 16

 9.1 Monitoring and alerting 16

 9.2 Backup and recovery 16

 9.3 Deployment strategy 16

10. Risks and Mitigations 17

1. Architecture Overview

1.1 System description

The Student Learning Management System (LMS) is a full-stack web platform enabling educational institutions to manage courses, content, assessments, and student progress. It supports three user roles — Admin, Instructor, and Student — and is built on AWS using Java Spring Boot microservices, a React/Next.js frontend, and a polyglot data tier.

1.2 Architecture style

The LMS adopts a microservices architecture with eight independently deployable Spring Boot services, each owning its domain data. Services communicate synchronously over REST via API Gateway for real-time operations and asynchronously via SNS/SQS for event-driven workflows. The frontend is a React SPA served via CloudFront CDN.

1.3 Architecture layers

Layer	Components	Responsibility
Client	React/Next.js SPA	Responsive UI for all three user roles — dashboard, video player, quiz interface, search
Edge	CloudFront · WAF + Shield · ACM · Route 53	CDN delivery, DDoS protection, rate limiting, TLS termination, DNS
API	Amazon API Gateway (REST + WebSocket)	Single entry point, JWT validation, throttling, WebSocket for real-time notifications
Application	8 x ECS Fargate Spring Boot services	Business logic — Auth, User, Course, Enrolment, Assessment, Progress, Notification, Logging
Messaging	SNS · SQS · EventBridge · Kinesis	Async decoupling, event fanout, scheduled triggers, high-volume log ingestion
Processing	AWS Lambda (5 functions)	Auto-grading, progress calculation, certificate generation, deadline reminders, log ingestion
Data	RDS PostgreSQL · Redis · DynamoDB · S3	Persistent storage, caching, event store, object store
Analytics	Athena · OpenSearch	Ad-hoc queries on log data, full-text log search, grade reports
Notification	SES · API GW WebSocket · Cognito	Email delivery, real-time in-app push, optional OAuth 2.0 identity
DevOps	CodePipeline · CodeBuild · ECR · CloudWatch · X-Ray · SonarQube · Amazon Q	CI/CD pipeline, Docker registry, observability, code quality, GenAI assistance
Security	IAM · VPC · Secrets Manager · KMS · WAF	Identity, network isolation, secrets rotation, encryption at rest and in transit

2. AWS Component Reference

2.1 Compute

Service	Configuration	Purpose
Amazon ECS Fargate	8 task definitions; min 1 task/service; auto-scaling on CPU > 60%; deployed across 2 AZs	Hosts all 8 Spring Boot microservices as Docker containers; no EC2 management needed
AWS Lambda	5 functions; 512 MB memory; 15-min timeout; triggered by SQS, SNS, EventBridge	Serverless async processing: auto-grade, progress calc, cert gen, deadline reminders, log ingestion
Amazon EC2 (none)	Not used in production	ECS Fargate is the compute layer; EC2 avoided to reduce operational overhead

2.2 Networking and edge

Service	Configuration	Purpose
Amazon Route 53	Hosted zone for lms.example.com; health-check routing; alias records to CloudFront	DNS management and health-based routing
Amazon CloudFront	Origins: S3 (assets) + ALB (API); WAF attached; ACM cert; gzip + Brotli compression	CDN for static assets and video streaming; satisfies 30s content load SLA (NFR001)
AWS WAF + Shield	Rate-based rules; AWS managed rule groups (OWASP Top 10); IP reputation lists	DDoS protection, rate limiting (NFR001), SQL injection and XSS prevention
Application Load Balancer	HTTPS listener; target groups per ECS service; /actuator/health health checks	Load distribution across ECS task replicas; SSL termination for internal traffic
Amazon VPC	1 VPC; 2 AZs; public subnet (ALB, NAT GW); private subnet (ECS); isolated subnet (data)	Network isolation; ECS services have no public IPs; data stores unreachable from internet
NAT Gateway	1 per AZ (2 total)	Allows ECS tasks in private subnet to reach internet (for SES, S3, SNS API calls)

2.3 Messaging and eventing

Service	Configuration	Purpose
Amazon SNS	Topics: grade-released, enrolment-confirmed, course-completed, deadline-reminder	Pub/sub fanout — one event triggers multiple consumers (email + in-app notification)

Service	Configuration	Purpose
Amazon SQS	Queues: auto-grade-queue, progress-queue, log-queue; DLQ on each; visibility timeout 30s	Async task buffering; absorbs submission and log write bursts; decouples request path from processing
Amazon EventBridge	Scheduled rules: deadline-reminder (daily), progress-refresh (configurable interval)	Time-based triggers for 48-hour deadline reminders (F009) and auto-refresh (F001)
Amazon Kinesis Data Streams	1 shard; 24-hour retention; consumer: log-ingestion Lambda	High-volume lesson view events buffered before writing to DynamoDB — prevents write bottleneck

2.4 Data stores

Service	Configuration	Rationale
Amazon RDS PostgreSQL 15	Multi-AZ; db.t3.medium (scalable to r6g.large); automated backups 7 days; read replica for reporting; Performance Insights enabled	ACID transactions for users, courses, enrolments, assessments; complex joins; FK integrity
Amazon ElastiCache Redis 7	Cluster mode; 2 shards x 2 replicas; encryption at rest and in transit; eviction policy: allkeys-lru	10-minute course metadata cache (F002); JWT session store; notification preference store
Amazon DynamoDB	On-demand capacity mode; TTL on Logs (90-day retention); PITR enabled; global tables optional	High write throughput for logs and progress events; no table locking; scales automatically with lesson view spikes
Amazon S3	Versioning enabled; SSE-S3 encryption; lifecycle: Glacier after 1 year; 3 buckets: content, submissions, certificates	Stores course videos/PDFs, assignment file uploads (F006), and generated PDF certificates (F007)

2.5 Security services

Service	Configuration	Purpose
AWS IAM	Task execution roles per ECS service; least-privilege policies; no wildcard actions	Identity and access management for all AWS service interactions
AWS Secrets Manager	DB credentials, JWT secret, SES API key; automatic rotation every 30 days	Secure secrets storage — no hardcoded credentials in code or environment variables
AWS KMS	Customer managed keys (CMK) for RDS, S3, DynamoDB encryption	Encryption key management with automatic annual rotation

Service	Configuration	Purpose
AWS AzCM	TLS certificates for CloudFront and ALB; auto-renewed	Managed TLS — no manual certificate handling
Security groups	sg-alb (443 inbound); sg-ecs (8080 from ALB only); sg-rds (5432 from ECS only); sg-redis (6379 from ECS only)	Network-level micro-segmentation between layers

2.6 DevOps and observability

Service	Configuration	Purpose
AWS CodePipeline	10-stage pipeline: source → lint → unit test → integration test → build → push → quality gate → staging deploy → smoke test → prod deploy	End-to-end CI/CD with automated quality gates (F011, NFR001)
AWS CodeBuild	Docker builds; JUnit + JaCoCo coverage reports; fail if coverage < 80%	Automated build and test execution
Amazon ECR	Private repository per service; image scanning on push; lifecycle policy (last 10 images retained)	Docker image registry
Amazon CloudWatch	Custom dashboards; alarms on error rate, CPU, memory, SQS depth; log groups per service	Metrics, alarms, and centralised log aggregation
AWS X-Ray	Instrumented in all Spring Boot services and Lambda functions	Distributed tracing — correlates requests across microservices
SonarQube	Quality gate: 0 blockers, 0 criticals, coverage >= 80%; blocks CodePipeline promotion on failure	Static analysis and coverage gate (F011)
Amazon Q Developer	IDE plugin for engineers; generates unit tests and boilerplate code	GenAI code companion to accelerate development (F001, F011)

3. Microservice Catalogue

Service	Port	Domain	Key dependencies	Publishes events
Auth service	8081	Identity	Users table (RDS), Redis (token store)	login-event → Logging
User service	8082	User mgmt	Users table (RDS)	user-updated → Logging
Course service	8083	Content	Courses, Modules, Lessons (RDS), S3 (content URLs), Redis (metadata cache)	course-published → Notification
Enrolment service	8084	Enrolment	Enrollments (RDS), Payment gateway	enrolment-confirmed → SNS
Assessment service	8085	Assessment	Assessments, Submissions (RDS), S3 (file uploads), SQS (auto-grade queue)	submission-created → SQS
Progress service	8086	Progress	Enrollments (RDS), DynamoDB (events), SNS	course-completed → SNS
Notification service	8087	Communications	Redis (preferences), SES, API GW WebSocket	(consumes SNS events)
Logging service	8088	Audit	DynamoDB (logs), OpenSearch (index)	(consumes all service events)

4. Key Data Flows

4.1 Student submits a quiz (happy path)

Step	From	To	Protocol	Notes
1	Student browser	CloudFront / WAF	HTTPS	POST /api/submissions with JWT
2	API Gateway	Auth service	Internal REST	JWT token validation
3	API Gateway	Assessment service	Internal REST	Forward validated request
4	Assessment service	RDS PostgreSQL	SQL	Check attempt count, save submission (score=NULL)
5	Assessment service	SQS auto-grade queue	AWS SDK	Enqueue grading job (async)
6	Assessment service	Student browser	HTTPS	202 Accepted response
7	SQS	Auto-grade Lambda	Event trigger	Lambda invoked with submission payload
8	Lambda	RDS PostgreSQL	SQL	Load questions, compute score, update submission
9	Lambda	Progress service	Internal REST	Notify lesson/assessment complete
10	Progress service	SNS grade-released	AWS SDK	Publish grade event if score changed
11	SNS	Notification service	SQS subscription	Fan out grade notification
12	Notification service	Amazon SES	AWS SDK	Send grade email to student
13	Notification service	API GW WebSocket	WebSocket	Push in-app real-time notification

4.2 Certificate generation flow

Step	Trigger	Action	Result
1	Progress service detects CompletionPercentage = 100	Publish course-completed event to SNS	Event routed to cert-gen Lambda
2	Cert-gen Lambda invoked	Load student and course details from RDS	Data retrieved for PDF generation

Step	Trigger	Action	Result
3	Lambda generates PDF	Use HTML-to-PDF library (iText / Flying Saucer)	Professional certificate PDF
4	Lambda uploads to S3	PUT to s3://lms-certificates/{studentId}/{courseId}.pdf	Certificate stored durably
5	Lambda updates RDS	UPDATE enrollments SET certificateUrl = ... WHERE enrollmentId = ...	URL recorded on enrolment record
6	Lambda publishes notification	SNS certificate-ready event	Student notified via email + WebSocket

5. Architecture Design Decisions

The following table documents the key decisions made during architecture design. These are the primary discussion points for the Architecture Review Board.

Decision	Options considered	Choice	Rationale
Container orchestration	ECS Fargate vs EKS vs EC2 Auto Scaling	ECS Fargate	No cluster management; serverless containers; lower ops overhead; EKS adds Kubernetes complexity not justified at this scale
Database strategy	Single RDS vs polyglot persistence	Polyglot: RDS + DynamoDB + Redis + S3	Each data type has different access patterns — relational for ACID, DynamoDB for high-write logs, Redis for low-latency cache, S3 for binary objects
Video delivery	Store on EC2 vs S3 + CloudFront	S3 + CloudFront + pre-signed URLs	Satisfies 30s load SLA; pre-signed URLs bypass API tier for direct browser-to-CDN streaming; infinite scale; no egress through app servers
Async processing	Synchronous in-service vs Lambda via SQS	Lambda async	Decouples slow ops (grading, cert gen) from API response path; improves P95 latency; scales independently; DLQ for failure isolation
Log store	RDS PostgreSQL vs DynamoDB	DynamoDB	Every lesson view fires a log entry — RDS would become a write bottleneck; DynamoDB on-demand scales automatically; TTL for retention
Authentication	AWS Cognito vs custom JWT service	Custom Spring Boot Auth service	Full control over RBAC claims; Cognito noted as optional v2 enhancement for OAuth 2.0 / social login
Notifications	Polling vs WebSocket vs SSE	WebSocket (API GW)	Bi-directional real-time; managed WebSocket in API GW removes server connection state; polling rejected for latency and server load
API style	REST vs GraphQL vs gRPC	REST + WebSocket	REST is well-understood and sufficient; WebSocket added only for real-time notifications; gRPC overhead not justified for public-facing API
CI/CD	Manual deploy vs CodePipeline	AWS CodePipeline + CodeBuild	Automated quality gates (SonarQube, 80% coverage) enforced on every merge; blue/green deploy with auto-rollback

Decision	Options considered	Choice	Rationale
Frontend hosting	EC2/ECS vs S3 + CloudFront	S3 + CloudFront	Static SPA served from edge; near-zero latency for assets; infinite scale; no server to maintain for frontend tier

6. Non-Functional Requirements Compliance

NFR	Requirement	Architecture approach	Status
Error handling	Comprehensive throughout application	@ControllerAdvice GlobalExceptionHandler in every service; RFC 7807 error format; CloudWatch alarms on error rate; X-Ray traces on exceptions	Addressed
Rate limiting	Prevent API abuse	AWS WAF rate-based rules (100 req/min per IP unauthenticated, 300 req/min authenticated); API Gateway throttling; 429 + Retry-After response	Addressed
Content load time	Load within 30 seconds	CloudFront CDN at edge; S3 pre-signed URLs bypass API tier for video; ElastiCache Redis 10-min metadata cache; CloudFront P95 < 2s for assets	Addressed
Containerisation	Docker for deployment	All 8 Spring Boot services containerised; Docker images built by CodeBuild; stored in ECR; deployed to ECS Fargate	Addressed
Modular codebase	Modular with CI/CD and automated tests	Microservices enforce modularity; each service has independent pipeline; SonarQube quality gate; 80% coverage gate; JUnit + Testcontainers; Postman smoke tests	Addressed

7. Scalability and High Availability

7.1 Scalability approach

Component	Scaling mechanism	Trigger
ECS Fargate services	Application Auto Scaling — add/remove tasks	CPU > 60% sustained for 3 minutes
RDS PostgreSQL	Vertical scaling (resize instance class); read replica for reporting queries	Manual during maintenance window
ElastiCache Redis	Cluster mode — horizontal sharding; add shards via online resizing	Manual based on memory usage
DynamoDB	On-demand capacity mode — scales automatically with no config	Automatic — no trigger needed
Lambda	Concurrent executions scale automatically (up to account limit)	Per-request — automatic
S3 + CloudFront	Unlimited scale inherently	No scaling action needed
SQS	Queue depth triggers Lambda concurrency scaling	SQS depth > configured threshold

7.2 High availability

- All ECS services deployed across 2 Availability Zones (AZ-1a and AZ-1b) with minimum 1 task per AZ
- RDS Multi-AZ: synchronous replication to standby; automatic failover within 60–120 seconds
- ElastiCache Multi-AZ replication group: automatic failover to read replica on primary failure
- S3: 99.999999999% (11 nines) durability; cross-region replication for content bucket
- CloudFront: origin failover group; automatic switch to S3 if ALB origin unhealthy
- ALB health checks: mark ECS tasks unhealthy and stop routing if /actuator/health returns non-200
- DynamoDB: Multi-AZ by default; PITR for point-in-time recovery up to 35 days

8. Security Architecture

8.1 Defence in depth layers

Layer	Control	Implementation
Network edge	DDoS and rate limiting	AWS Shield Standard + WAF managed rule groups + rate-based rules
Transport	TLS encryption	ACM certificates on CloudFront and ALB; TLS 1.2+ enforced; HTTP → HTTPS redirect
Authentication	JWT token validation	API Gateway Lambda authoriser validates token on every request before forwarding to services
Authorisation	RBAC	@PreAuthorize Spring Security annotations in every controller; role embedded in JWT claim
Application	Input validation	Bean Validation (@Valid) on all request bodies; OWASP WAF rules for SQL injection / XSS
Data at rest	Encryption	RDS: AES-256 via KMS CMK; S3: SSE-S3; DynamoDB: AWS-managed encryption; ElastiCache: encryption at rest
Secrets	Secrets management	All credentials in AWS Secrets Manager; auto-rotated every 30 days; no hardcoded secrets
Passwords	Hashing	BCrypt with cost factor 12 before storing in RDS (F003)
Network isolation	VPC segmentation	ECS in private subnet; RDS/Redis in isolated subnet; no public IPs on app or data tiers
Audit	Event logging	Every user action logged to DynamoDB via Logging service (F010); Admin filterable

9. Operational Considerations

9.1 Monitoring and alerting

Metric	Alarm threshold	Action
API Gateway 5xx rate	> 1% over 5 minutes	PagerDuty alert → on-call engineer
ECS CPU utilisation	> 80% sustained	Auto-scaling triggers; alert if max capacity reached
RDS connections	> 80% of max_connections	Alert + investigate connection pool leaks
SQS queue depth	> 1000 messages for > 10 minutes	Alert — Lambda processing may be backlogged
ElastiCache evictions	> 0 per minute sustained	Alert — cache too small; scale up
Lambda error rate	> 5% over 15 minutes	Alert + review DLQ for failed messages
CloudFront 4xx rate	> 5% over 5 minutes	Investigate — may indicate auth issues or bad client requests

9.2 Backup and recovery

Resource	Backup method	RPO	RTO
RDS PostgreSQL	Automated daily snapshots (7-day retention) + continuous PITR	< 5 minutes	< 30 minutes (restore to new instance)
DynamoDB	PITR enabled (35-day window)	< 5 minutes	< 1 hour (restore to new table)
S3 content	Versioning enabled; cross-region replication	Near-zero (replication lag < 15 min)	Immediate (switch to replica bucket)
ElastiCache Redis	Daily snapshots to S3	< 24 hours	< 30 minutes (restore from snapshot)

9.3 Deployment strategy

All ECS service deployments use blue/green deployment via AWS CodeDeploy. The new task set (green) is started alongside the existing one (blue). Traffic is shifted 10% at a time over 10 minutes. If any CloudWatch alarm fires during the shift, deployment is automatically rolled back to the blue environment within 5 minutes. Zero-downtime deployments are guaranteed.

10. Risks and Mitigations

Risk	Likelihood	Impact	Mitigation
Video streaming exceeds 30s SLA on slow connections	Medium	High	Adaptive bitrate streaming (HLS); CloudFront edge caching; content quality tiers
DynamoDB hot partition on high-traffic log writes	Low	Medium	Partition key design: actorId#date prefix distributes writes; on-demand mode absorbs spikes
JWT token compromise	Low	High	Short access token TTL (15 min); refresh token rotation; Redis token blacklist on logout
Manual grading SLA breach	Medium	Medium	Instructor dashboard alerts on ungraded submissions > 48 hrs; Admin escalation report
ECS cold start latency on new task spin-up	Low	Low	Minimum 1 task always running per service; pre-warm endpoint via EventBridge scheduled ping
SonarQube gate blocking releases	Medium	Low	Warning-only mode for first 2 sprints; hard gate enforced from sprint 3 onwards